

LEARNMATE PROJECT

Demo Executable / Demonstration Guide

1. Project Overview

This PDF contains the demo runner and demonstration structure prepared from the supplied Learnmate project screenshots. The visible project uses Node.js/Express, MongoDB through Mongoose, JWT authentication, bcrypt password hashing, Multer file uploads, and Gemini-based AI content generation.

2. Project Structure

```
Code Files/
  src/
    controllers/
      adminController.js
      authController.js
      materialController.js
    middleware/
      auth.js
      upload.js
    models/
      Material.js
      User.js
    routes/
      admin.js
      auth.js
      materials.js
    utils/
      db.js
      gemini.js
      tokens.js
  uploads/
  .env
  index.js
  package.json
  README.md
```

3. Demonstration Flow

Step	Demo
1	Start the Learnmate Express backend.
2	Verify the API health endpoint.
3	Demonstrate user authentication using JWT Bearer tokens.
4	Demonstrate study material handling.
5	Demonstrate AI-generated summary, flashcards, quiz and study plan.

4. Demo File — demo.js

```
/**
 * Learnmate - Project Demo Runner
 * -----
 * This file is designed to demonstrate the backend shown in the project screenshots:
 * - Express API health check
 * - JWT-based authentication flow
 * - Material/study-content flow
 * - AI content generation concept using Gemini
 */
```

```

* It does NOT contain real API keys or passwords.
*
* Usage:
*   1. Start the backend: node index.js
*   2. In another terminal: node demo.js
*
* Environment overrides:
*   DEMO_BASE_URL=http://localhost:5000 node demo.js
*
* IMPORTANT:
* The exact controller route bodies are project-specific. The health check is
* guaranteed by the supplied index.js. The other demo requests are written as
* clearly marked examples and can be enabled after matching your controller
* request/response format.
*/

```

```
require("dotenv").config();
```

```
const BASE_URL = process.env.DEMO_BASE_URL || `http://localhost:${process.env.PORT || 5000}`;
```

```
const sampleMaterial = {
  title: "Introduction to Artificial Intelligence",
  content:
    "Artificial Intelligence is the field of building systems that can perform tasks " +
    "that normally require human intelligence, such as learning, reasoning and problem solving."
};
```

```

async function request(path, options = {}) {
  const response = await fetch(`${BASE_URL}${path}`, {
    ...options,
    headers: {
      "Content-Type": "application/json",
      ...(options.headers || {})
    }
  });
};

const text = await response.text();
let body;
try {
  body = JSON.parse(text);
} catch {
  body = text;
}

return { status: response.status, ok: response.ok, body };
}

```

```

async function main() {
  console.log("\n=====");
  console.log("          LEARNMATE PROJECT DEMO");
  console.log("=====\\n");
  console.log(`Backend URL: ${BASE_URL}\\n`);

  // 1. Backend health check
  console.log("[1/4] Checking backend...");
  try {
    const health = await request("/");
    console.log(`Status: ${health.status}`);
    console.log("Response:", health.body);
  } catch (error) {
    console.error("Backend is not reachable.");
    console.error("Start it first with: node index.js");
    console.error(error.message);
    process.exitCode = 1;
    return;
  }

  // 2. Show the demo material that can be used for upload/material creation.
}

```

```

console.log("\n[2/4] Sample learning material");
console.log("-----");
console.log(JSON.stringify(sampleMaterial, null, 2));

// 3. Authentication request template.
console.log("\n[3/4] Authentication flow");
console.log("-----");
console.log("The supplied project uses JWT Bearer authentication.");
console.log("Expected flow:");
console.log("  Register/Login -> receive access token ->");
console.log("  Authorization: Bearer <token> -> protected API");

// 4. AI study-content demo
console.log("\n[4/4] AI study-content flow");
console.log("-----");
console.log("Gemini is configured in src/utils/gemini.js.");
console.log("The project can use the uploaded material to generate:");
console.log("  • Summary");
console.log("  • Flashcards");
console.log("  • Quiz questions");
console.log("  • Study plan");

console.log("\n=====");
console.log("Demo structure verified.");
console.log("=====\n");
}

main().catch((error) => {
  console.error("Demo failed:", error);
  process.exitCode = 1;
});

```

5. Sample AI Study Material

Title: Introduction to Artificial Intelligence

Summary: AI enables computer systems to perform tasks associated with human intelligence.

Flashcards

Q: What is Artificial Intelligence?

A: A field that creates systems capable of performing tasks associated with human intelligence.

Q: Give two examples of AI tasks.

A: Learning and problem solving.

Quiz

Question: Which is an example of an AI task?

Options: Learning | Printing paper | Charging a battery | Turning on a switch

Answer: Learning

Study Plan: Review the definition, learn the main applications, then practise with the flashcards and quiz.

6. Run Commands

```
# Terminal 1 – start backend
node index.js

# Terminal 2 – run demo
node demo.js

# If the backend uses another port
DEMO_BASE_URL=http://localhost:5000 node demo.js
```

7. Important Note

The supplied screenshots do not show the complete implementations of `authController.js` and `materialController.js`. Therefore, this demo does not invent their exact request payloads or response formats. Those endpoints should be connected to the actual controller definitions before presenting automated API requests.

Keep `MONGO_URI`, `JWT_ACCESS_SECRET` and `GEMINI_API_KEY` in the project's `.env` file. Do not place real credentials inside the demo file.